

A Tool for Digital Circuits Synthesis Based on Notification Oriented Paradigm

R. Kerschbaumer, J. M. Simão, J. A. Fabro C. R. Erig Lima and R. R. Linhares

Abstract—FPGAs (Field Programmable Gate Arrays) are proving to be an alternative for applications that require great processing power because they allow parallelism. Still, the high-level synthesis tools allow professionals without hardware knowledge to use FPGAs. However, this high-level tools do not allow to properly explore the parallelism because they are either based on sequential techniques or rely on languages with an insufficient level of abstraction. In this paper it is proposed the use of the Notification Oriented Paradigm (NOP) for Digital Hardware as a high-level synthesis tool. The NOP is a development technique that presents a new concept for designing and executing applications, based on rules and collaborative entities, which communicate through notifications in a parallel way. In order to allow the use of the NOP in the creation of digital hardware as a new high-level synthesis tool, a new programming language was proposed, and both a compiler and an emulator were implemented. With these tools it is possible to generate VHDL code directly from a high level NOP program. In order to evaluate the performance and stability of circuits generated with these tools, some experiments were performed. These experiments demonstrated that the NOP for Digital Hardware allows quickly and easily creating reliable digital circuits with suitable performance and parallelism, directly from a high-level development environment.

Keywords—FPGA, Notification Oriented Paradigm, VHDL.

I. INTRODUÇÃO

COM a crescente importância da computação em várias áreas, a demanda por mais poder de processamento está crescendo rapidamente. Arquiteturas de computação tradicionais estão tendo dificuldades em acompanhar esse crescimento [1][2]. A necessidade de novas arquiteturas de computação é evidente e as FPGAs (Field Programmable Gate Arrays ou Matrizes de Portas Programáveis em Campo) vem se mostrando uma opção [1][2][3]. Os fabricantes de microprocessadores vêm demonstrando crescente interesse nas FPGAs, pois sua natureza reconfigurável, seu baixo consumo de energia e sua grande capacidade de processamento permitem melhorar várias categorias de sistemas computacionais [4].

Se por um lado as FPGAs apresentam características de performance promissoras, por outro lado, sua programação é

tarefa complexa, normalmente exigindo conhecimentos avançados de *hardware* digital e de linguagens específicas de programação como VHDL ou Verilog [5]. Uma forma alternativa de aproveitar o potencial das FPGAs é a utilização de ferramentas de síntese de alto nível [6]. Estas ferramentas permitem que se programe em linguagens de mais alto nível. Algumas destas ferramentas utilizam linguagens derivadas do C e do C++, facilitando assim o reaproveitamento de códigos já existentes [6], mas ao mesmo tempo dificultando a execução em paralelo, que é uma das características mais importantes das aplicações em FPGAs. A utilização das ferramentas de alto nível também tem outras desvantagens, muitas vezes os circuitos gerados por estas ferramentas são mais lentos e ocupam mais recursos na FPGA do que os circuitos criados diretamente em linguagens como VHDL e Verilog [6][7].

Neste artigo é apresentada uma nova forma de criar circuitos digitais para FPGAs utilizando o Paradigma Orientado a Notificações (PON) [8][9]. O PON é utilizado em diferentes plataformas de computação para reduzir problemas de redundância temporal e redundância estrutural, bem como para permitir uma abstração de mais alto nível na forma de regras. O PON também é utilizado para viabilizar a execução paralela em plataformas que forneçam este suporte. Uma versão específica do PON para *Hardware* Digital (PON-HD) foi desenvolvida para ser utilizada como uma ferramenta de síntese de alto nível [10]. Para este fim foi criada uma linguagem de programação específica para o PON-HD. Também foi criado um compilador capaz de compilar esta linguagem gerando código VHDL, que por sua vez pode ser utilizado na geração de circuitos digitais nas FPGAs. Para verificar o comportamento dos programas criados com a ajuda do PON-HD foi desenvolvido um emulador. Este emulador permite testar a aplicação em um ambiente gráfico de fácil utilização.

Com o objetivo de verificar as características de desempenho e estabilidade (capacidade de manter seu funcionamento estável por longos períodos de tempo) dos circuitos gerados com a ajuda do PON-HD foram realizados vários experimentos, dentre os quais três serão descritos. O primeiro experimento foi um circuito contador e serviu para verificar o correto funcionamento das ferramentas. O segundo experimento foi um circuito ordenador de dados. Neste experimento foi possível variar o número de elementos a serem ordenados e desta forma verificar o comportamento do PON-HD para circuitos de tamanhos diferentes. Foram realizadas comparações de desempenho entre versões deste algoritmo de ordenação em PON-HD e VHDL. O último experimento é um circuito com a finalidade de controlar um robô hexápode. Neste experimento foi possível comprovar a

R. Kerschbaumer, Instituto Federal de Educação Ciência e Tecnologia Catarinense (IFC), Luzerna, Santa Catarina, Brasil, ricardo@luzerna.ifc.edu.br

J. M. Simão, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, jeansimao@utfpr.edu.br

J. A. Fabro, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, fabro@utfpr.edu.br

C. R. Erig Lima, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, erig@utfpr.edu.br

R. R. Linhares, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, linhares@utfpr.edu.br

Corresponding author: Ricardo Kerschbaumer

estabilidade dos circuitos gerados com o PON-HD. Os resultados mostraram que o PON-HD é uma alternativa interessante na criação de circuitos digitais, permitindo que se crie circuitos confiáveis e com bom desempenho de forma fácil e rápida.

O restante deste artigo está organizado com descrito a seguir. A seção II apresenta alguns conceitos importantes sobre ferramentas de síntese de alto nível. A seção III apresenta os fundamentos do PON e a seção IV apresenta em detalhes o PON-HD. O ambiente de desenvolvimento do PON-HD é apresentado na seção V, e os experimentos realizados são descritos na seção VI. A seção VII apresenta os resultados e finalmente a seção VIII apresenta as conclusões.

II. FERRAMENTAS DE SÍNTESE DE ALTO NÍVEL

As ferramentas de síntese de alto nível visam preencher a lacuna entre a concepção de algoritmos a serem executados diretamente em *hardware* e o projeto de *hardware* em si. Em suma, a linguagem de entrada e a facilidade de aplicação destas ferramentas permitem que o projeto de *hardware* seja executado por desenvolvedores sem experiência com projeto de *hardware* [6]. O processo de verificação, que é uma tarefa importante e complexa nos projetos atuais, também é facilitada pelo uso de ferramentas de síntese de alto nível [6].

Uma ferramenta de síntese de alto nível de boa qualidade deve utilizar linguagem compatível com as linguagens de projeto de algoritmos e deve também permitir a captura do projeto com certo nível de abstração. O tamanho do projeto gerado e sua latência após a síntese devem ser razoáveis. Ainda, é igualmente importante que a ferramenta e seus recursos sejam fáceis de usar [6].

A complexidade do trabalho realizado pelas ferramentas de síntese de alto nível está relacionada com a natureza paralela das operações realizadas nas FPGAs. Este paralelismo supostamente natural das operações permite transferir tarefas computacionais complexas que requerem movimentação de grandes volumes de dados e alto poder de processamento dos microprocessadores para as FPGAs. No entanto, a complexidade de implementação dos algoritmos nas FPGAs, devido justamente ao seu paralelismo, é maior do que nos microprocessadores sequenciais, exigindo assim mais esforço das ferramentas e dos desenvolvedores [5].

Neste âmbito, as linguagens de programação tradicionais não são adequadas para o desenvolvimento de projetos envolvendo paralelismo [9][11]. Assim, tais linguagens tradicionais não seriam ideais para aplicações que visem explorar as potencialidades das FPGAs [12]. Não obstante, grande parte da comunidade desenvolvedora segue usando ferramentas de síntese baseadas em linguagens usuais.

Neste contexto, a adoção de linguagens baseadas em C traz um bom nível de abstração, facilitando a adaptação dos algoritmos existentes e permitindo uma curva de aprendizado mais rápida [12]. Entretanto, as linguagens baseadas em C não são as mais apropriadas para as ferramentas de síntese de alto nível, devido principalmente à falta de informação temporal no código e a falta de um modelo de concorrência/paralelismo

fino [12].

Devido a esta questão, existem outras linguagens, cada vez mais usadas para essa finalidade. São exemplos de tais linguagens VHDL, Verilog, SystemC e Bluespec [12]. No entanto, essas linguagens dificultam a reutilização ou a adaptação de algoritmos originalmente escritos em linguagens como C e C++ [12]. Ademais, essas linguagens são de uso mais difícil que as tradicionais, pois obrigam o desenvolvedor a estar ciente do paralelismo e expressá-lo explicitamente.

Assim, o ideal seria linguagens de altíssimo nível que abstraíssem o paralelismo, gerando-o automaticamente para o desenvolvedor. Entretanto, até agora não há uma solução definitiva para tornar mais fácil a programação de FPGAs [12]. Na verdade, há uma variedade de linguagens com características melhores ou piores conforme o contexto e a aplicação visada. Como as ferramentas de síntese de alto nível variam muito em relação à linguagem de entrada, é importante escolher a ferramenta adequada para cada projeto [12].

É possível classificar as ferramentas de síntese de alto nível por suas características. As principais características de uma ferramenta de síntese de alto nível são: facilidade de implementação, nível de abstração, tipos de dados, independência de arquitetura, capacidade de verificação, qualidade dos resultados, documentação e curva de aprendizado [6].

É ainda importante medir a qualidade dos resultados das ferramentas de síntese de alto nível. Para a avaliação da qualidade de um *hardware* gerado por uma ferramenta de síntese de alto nível, três métricas podem ser utilizadas: a máxima frequência de operação, que é a máxima frequência em MHz em que um circuito pode operar; a latência, que é o número de ciclos de *clock* necessários para completar uma tarefa; e o uso de memória, interna ou externa [7].

III. O PARADIGMA ORIENTADO A NOTIFICAÇÕES

Os principais paradigmas de programação atuais podem ser classificados como imperativo e declarativo [13]. No imperativo o código é escrito de forma a definir sequencialmente as atividades a serem realizadas pelo dispositivo computacional, inclusive aquelas de decisão ou inferência [13]. No paradigma declarativo o código é declarado em nível mais alto que no imperativo, sendo a sequência de execução ditada por uma máquina de inferência explícita. Outrossim, tanto as abordagens imperativas quando as declarativas apresentam um conjunto de imperfeições [13].

O Paradigma Imperativo (PI) pode ser entendido como compreendendo as abordagens chamadas procedimental e as abordagens orientadas a objetos. Estas abordagens, em suma, apresentam tendência a muita redundância de código, acoplamento das partes e miscelânea dos elementos tais como elementos executacionais (e.g. escrita e leitura), elementos lógico-causais para decisão ou inferência elementar (e.g. se então) e elementos factuais/dados (e.g. variáveis e vetores). Ainda que mitigados um tanto mesmo quando boas práticas de programação são utilizadas, precedidas das fases apropriadas de engenharia de software, esses problemas tendem a ocorrer. Ademais, essas práticas exigem

conhecimentos mais avançados do desenvolvedor. Exemplos de tais práticas são os usos de encapsulamento (coesão/desacoplamento), padrões de codificação e mesmo de projeto, e orientação a eventos [13]. Essas questões são tratadas em maiores detalhes em trabalhos como [14][15].

Por sua vez, o Paradigma Declarativo (PD) pode ser entendido como compreendendo as abordagens lógicas e abordagens funcionais. Essas abordagens do PD, em suma, apresentam acoplamento entre elementos factuais/execucionais (base de fatos) e elementos lógico-causais (base de regras) devido ao uso de máquinas de inferência baseadas em pesquisas sobre listas de elementos, as quais confrontam por percorrimto esses dois conjuntos de elementos (i.e. base de fatos e base de regras). Ainda, essas abordagens resultam em sobrecargas de processamento em suas soluções de inferência dado que, mesmo com algoritmos com otimizações de pesquisas, elas fazem uso de estruturas de dados computacionalmente dispendiosas [14][15].

Na verdade, os paradigmas de programação atuais são movidos por inferência monolítica implícita (no caso de PI) ou explícita (no caso de PD), que de alguma forma realiza pesquisas sobre elementos passivos em uma base de fatos (por exemplo, variáveis, objetos, vetores etc.) a fim de testar expressões lógico/causais (por exemplo, a instrução *if-then* ou regras semelhantes). Isto frequentemente resulta, tanto em PI quanto em PD, em acoplamento de código e redundâncias, o que implica em dificuldades na distribuição do processamento e sobrecargas, como detalhado em [9].

Neste contexto, uma nova técnica de programação, chamada Paradigma Orientado a Notificações (PON), foi proposta [8][9]. Os fundamentos do PON foram propostos inicialmente por J. M. Simão como uma solução de controle discreto de manufatura [9]. Esta solução evoluiu como uma solução geral de controle discreto [14]. Posteriormente, ela evoluiu como uma nova solução de inferência. Depois disso ainda, tal solução evoluiu como um paradigma ou técnica (conforme a compreensão destes termos) de programação e desenvolvimento. Por fim, mais recentemente, ela evoluiu para um paradigma computacional [8][9].

O PON, como paradigma de desenvolvimento, apresenta um novo conceito para conceber e executar aplicativos, com base em entidades-regras, compostas de sub-entidades colaborativas, as quais são notificáveis a partir de entidades factuais. A essência do PON é seu processo de inferência baseado em sub-entidades pequenas, inteligentes e desacopladas, que colaboram entre si através de notificações pontuais a fim realizar todo o cálculo lógico-causal [8][9].

O PON resolve problemas de redundância e centralização das abordagens atuais de processamento lógico-causal, resolvendo assim as questões de mau uso da capacidade de processamento e acoplamento dos paradigmas atuais [8][9]. O objetivo do PON é tornar mais fácil a tarefa de criação de aplicativos, os quais sejam melhores em termos de facilidade de composição de código otimizado e distribuível [9].

A. Estrutura e processo de inferência do PON

No PON, as expressões causais são representadas por

regras lógico-causais usuais, as quais são naturais aos desenvolvedores dos paradigmas atuais de programação. Tecnicamente, cada regra lógico-causal é tratada com uma entidade computacional especial chamada *Rule* [14][15]. Uma *Rule*, de uma regra lógico-causal, em um formato amigável ou em alto nível, é ilustrada na Fig. 1.

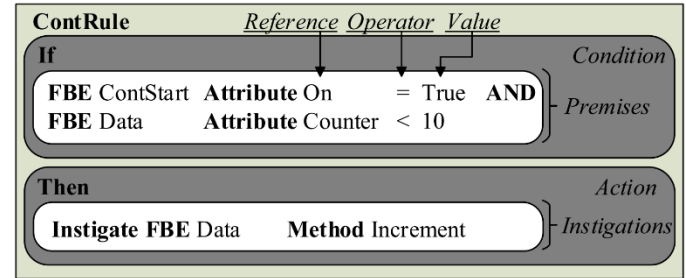


Figura 1. A representação de uma *Rule*.

Estruturalmente, cada *Rule* é composta por uma *Condition* e uma *Action*, como mostrado através do diagrama de blocos em SysML na Fig. 2. A *Condition* e a *Action* são entidades que trabalham juntas para representar e tratar o conhecimento causal da *Rule*. Cada *Condition* é responsável pela tomada de decisão na sua *Rule*, enquanto cada *Action* é responsável pela conclusão relativa a decisão positiva. Neste âmbito, a *Condition* avalia estados de elementos factuais, enquanto a *Action* instiga a execução de serviços desses. No exemplo apresentado, estes elementos são o *ContStart* e *Data*.

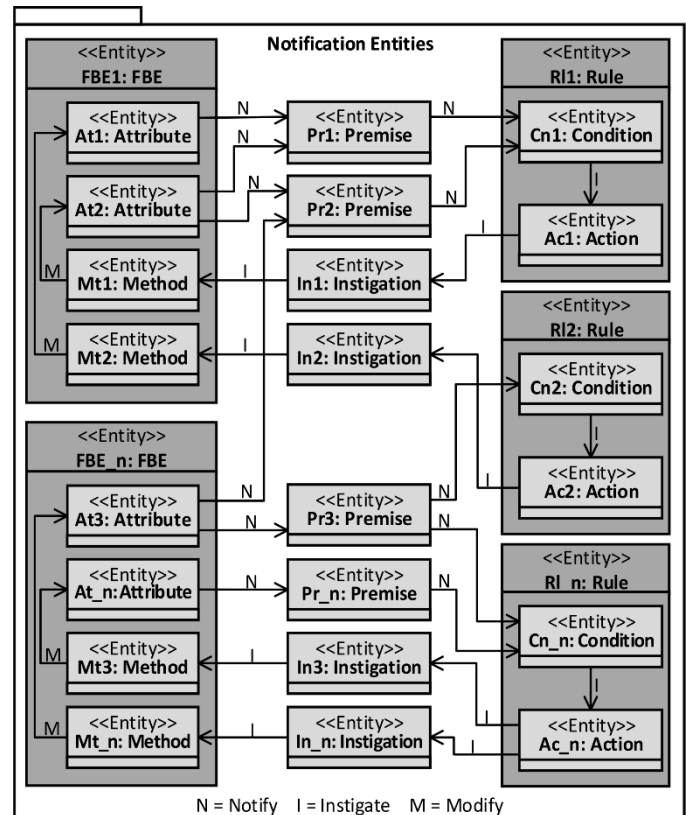


Figura 2. Diagrama de blocos em SysML do ciclo de notificações do PON.

Cada elemento avaliado no PON por *Rules* é representado por um tipo de entidade chamado *Fact_Base_Element* (FBE).

Um *FBE* é composto por um conjunto não vazio de atributos. Cada atributo é representado por um outro tipo de entidade chamada *Attribute*. No exemplo, *On* e *Counter* são respectivamente *Attributes* das *FBEs* *ContStart* e *Data*. Os estados dos *Attributes* são fatos analisáveis em um processo de inferência nas *Conditions* das *Rules*. Entretanto, isso se dá por meio de outras entidades chamadas *Premises*, como modelado na Fig. 2. Na *Rule* do exemplo (Fig. 1), a *Condition* é composta por duas *Premises*.

Uma vez que cada *Premise* de uma *Condition* é inferida como verdadeira, a *Condition* torna-se verdade aprovando sua *Rule*. Uma *Rule* aprovada pode ativar sua *Action*, a qual é composta por entidades chamadas *Instigations*. Na *Rule* do exemplo, a *Action* contém uma *Instigation*. Na verdade, *Instigations* instigam *Methods*. Por sua vez, cada *Method* é outra entidade da *FBE*. Cada *Method* permite a execução de serviços da *FBE*. Geralmente, a chamada de um *Method* de uma *FBE* altera os estados dos *Attributes* de uma *FBE*, alimentando assim o processo de inferência [14][15].

O processo de inferência do PON é diferenciado, uma vez que as *Rules* têm sua inferência efetuada pela colaboração ativa de suas entidades notificantes [8]. Em suma, a colaboração acontece da seguinte forma: para cada alteração no estado de um *Attribute* de uma *FBE*, a avaliação do estado deste *Attribute* ocorre apenas nas *Premises* relacionadas e então somente nas *Conditions* relacionadas e pertinentes, por meio de notificações pontuais entre os colaboradores [14][15].

B. Inferência Orientada a Notificações do PON

Para explicar o processo de inferência orientada a notificação, primeiramente é necessário explicar a natureza e a composição da *Premise*. Cada *Premise* lida com um valor booleano, relacionado com um ou mesmo dois estados de *Attributes*, sendo compostos por: (a) uma referência ao valor de um *Attribute*, que é chamado de *Reference* e recebido por notificação; (b) um operador relacional, que é chamado de *Operator* e é usado para fazer comparações; e (c) outro valor, que é chamado de *Value* e pode ser uma constante ou o valor de outro *Attribute*, recebido também por notificação [14][15].

Uma *Premise* faz um cálculo lógico quando recebe uma notificação de um ou de dois *Attributes* (ou seja, *Reference* ou *Value*). Este cálculo é realizado comparando-se a *Reference* e o *Value*, usando o *Operator*. De forma bastante semelhante, uma *Premise* colabora com a avaliação causal de um conjunto de *Conditions*. Se for alterado o valor booleano de uma *Premise* notificada, então as *Conditions* relacionadas também são notificadas.

Assim, cada *Condition* notificada calcula seu valor booleano por meio de um operador lógico, como uma conjunção ou uma disjunção, dos valores das *Premises* em questão. No caso de uma conjunção (operador *AND*), por exemplo, quando todas as *Premises* que integram a *Condition* são satisfeitas, a *Condition* em si é satisfeita. No caso de uma disjunção (operador *OR*), como outro exemplo, quando uma das *Premises* que integra a *Condition* é satisfeita, a *Condition* em si é satisfeita. Uma vez que a *Condition* é satisfeita, ela notifica a respectiva *Rule* para que seja aprovada. Cada *Rule*

aprovada que esteja sem conflito com outra *Rule* ou tenha esse conflito resolvido pode ser executada [14][15]. Para executar, a *Rule* notifica sua *Action*, a qual a seu turno notifica cada uma das suas *Instigations*. Por fim, cada *Instigation* notifica cada um de seus *Methods*, instigando-o a executar. A colaboração entre as entidades do PON por meio de notificações pode ser observada no diagrama de blocos ilustrado na Fig. 2.

Um ponto importante a esclarecer sobre as entidades colaborativas do PON é que cada notificador (por exemplo, um *Attribute*) leva em conta seus notificados (por exemplo, *Premises*) no momento da sua criação. Por exemplo, quando uma *Premise* é criada e faz referência a um *Attribute*, o último automaticamente leva em conta o anterior no seu conjunto interno de elementos para ser notificado quando seu estado for alterado [14][15].

C. A natureza do PON

No PON, cada entidade factual *Attribute* é avaliado por um conjunto de entidades lógicas e causais, ou seja, as *Premises* e as *Conditions* na mudança de seu estado. Graças à cooperação por meio de notificações pontuais entre essas entidades factuais, lógicas e causais, o PON evita os dois tipos de redundâncias verificadas em muitas linguagens de programação. Em tempo, tais redundâncias geram desperdício de processamento e mesmo acoplamento entre partes do código [14][15].

Estes dois tipos de redundâncias são a redundância temporal e a redundância estrutural. A redundância temporal consiste na avaliação desnecessária de expressões lógico/causais. Por exemplo, uma expressão dada “se ($A > 3$) então” não aprovada por valor atual da variável *A*, sendo reavaliada outra(s) vez(es) em um laço de repetição sem ter havido mudanças na variável *A*. A redundância estrutural, por sua vez, é a repetição de expressão lógica no âmbito de expressões causais [14][15]. Por exemplo, uma expressão lógica dada ($A > 3$) repetida no âmbito de duas ou mais expressões “se então”.

A redundância temporal é resolvida no PON eliminando a pesquisas sobre elementos passivos. Em primeiro lugar, os *Attributes* são reativos em relação a atualização de seu estado. Em segundo lugar, os *Attributes* notificam pontualmente apenas as *Premises* que estão interessadas na atualização de seu estado. Portanto, evita-se assim que outras partes e até mesmo outras expressões causais sejam avaliadas ou reavaliadas desnecessariamente [14][15].

A redundância estrutural é também resolvida no PON. Isso acontece quando uma *Premise* é compartilhada por duas ou mais *Conditions*. Assim, a *Premise* realiza o cálculo lógico apenas uma vez e compartilha o resultado lógico apenas com as *Conditions* relacionadas, evitando assim reavaliações desnecessárias e a repetição de *Premise* iguais. Desta forma, evitando redundâncias estruturais e, principalmente, evitando redundâncias temporais, é possível obter uma melhora no desempenho [14].

Além de auxiliar a resolver problemas de desempenho, o PON é também potencialmente aplicável no desenvolvimento

de aplicações paralelas ou distribuídas. Isto ocorre devido ao "desacoplamento" de suas entidades (ou acoplamento mínimo entre entidades para ser mais preciso) permitido pela orientação a notificações.

Em termos de inferência, não há grande diferença, se uma entidade é notificada na mesma região de memória, na memória do mesmo computador ou na mesma sub rede. Por exemplo, uma entidade notificante (e.g. um *Attribute*) pode estar em execução em uma máquina ou processador e a entidade notificável (e.g. uma *Premise*) pode estar em execução em outro. Para o notificador, é apenas necessário saber o endereço da entidade notificável [14][15].

D. As plataformas do PON

O PON já foi implementado em várias plataformas diferentes para apoiar o desenvolvimento e execução de aplicativos neste paradigma.

A primeira implementação foi um *framework* C++, com três diferentes versões principais. Estas versões foram ainda derivadas para lidar com diferentes ambientes, como aplicações *multi-thread*, plataformas *multi-core*, sistemas *fuzzy* e redes neurais. Além disso, versões do *framework* foram reconstruídas em outras linguagens de programação, como Java e C#. Assim, nas aplicações PON desenvolvidas nestes *frameworks*, os objetos atuam como entidades notificantes [16].

Estes *frameworks* permitiram demonstrar a viabilidade do PON e suas propriedades, como o nível de abstração da programação, a facilidade da distribuição do código e o desempenho razoavelmente adequado. No entanto, o desempenho não era tão bom como previsto na teoria do PON e a facilidade de implementação poderia também ser melhorada. Assim, também foi desenvolvida uma linguagem de programação (LingPON) e o respectivo compilador, que é capaz de gerar código para o *framework* C++, código específico em C, código específico em C++ e até mesmo código específico em C++ estático (utilizando elementos estáticos), tudo a partir de uma linguagem PON de alto nível. Existe ainda uma adaptação do compilador PON para sistemas *fuzzy*. Contudo, como os códigos gerados pelo compilador são *single-threaded*, eles executam sequencialmente o processo de notificação de PON. No entanto, o nível de abstração das implementações com o PON e o desempenho foram melhorados. Tanto a linguagem quanto o compilador estão em evolução buscando melhorias diversas [16].

Há também outros esforços para desenvolver uma arquitetura onde o PON é implementado parte em *hardware* e parte em software. A parte implementada em *hardware* é um co-processador que é responsável pelas avaliações lógico-causais. A parte em software é um programa que lida com o processamento factual e executacional e é processado em um núcleo Von Neumann comum. Uma implementação desta arquitetura foi realizada em lógica reconfigurável (FPGA) e reduziu significativamente o número de ciclos de *clock* necessários para executar as tarefas, quando comparado com o *Framework* C++ do PON [17].

Além disso, foi desenvolvida uma arquitetura de

computação PON (*Notification Oriented Computing architecture* - NOCA). Ela foi usada como base para implementar um microprocessador PON completo em uma FPGA. Esta implementação foi capaz de buscar em memória e executar aplicações PON desenvolvidas como um software de baixo nível, de forma semelhante a um processador Von Neumann comum, porém com uma dinâmica de execução (propagação de notificações) mais próxima à dinâmica definida pelo modelo teórico do PON [18].

Finalmente, a fim de lidar com aplicações realmente paralelas em *hardware* digital (HD), uma implementação do PON puramente no *hardware* digital (PON-HD) foi desenvolvida [10]. Nesta implementação, o desenvolvedor pode usar a expressividade do paradigma para o projeto da aplicação aproveitando o paralelismo de execução das FPGAs. Esta implementação foi objeto de um pedido de patente em 2012 [10].

A abordagem do PON-HD é usada nos experimentos apresentados neste trabalho. Portanto, a próxima seção apresenta o PON-HD em detalhes.

IV. O PON EM HARDWARE DIGITAL

A implementação do PON-HD apresentada neste trabalho é uma interpretação do que foi objeto do pedido de patente em 2012, bem como uma evolução da implementação precedente também relativa ao pedido em questão [10].

A. A implementação do PON-HD

Na implementação do PON em *hardware* digital, os elementos que constituem este paradigma foram implementados como componentes VHDL. No entanto, alguns dos componentes do PON servem para organizar e agrupar outros elementos e não possuem correspondentes físicos. A Fig. 3 apresenta o diagrama de blocos da Fig. 2 com os elementos do PON que não possuem correspondentes físicos no PON-HD marcados como *Conceptual*. Um exemplo é o FBE, que serve para agrupar *Methods* e *Attributes* e não tem função no *hardware* gerado.

A operação do PON-HD pode ser resumida em: (a) os valores dos *Attributes* são comparados pelas *Premises*, resultando em um valor lógico (verdadeiro ou falso); (b) se a operação lógica definida pela *Condition*, sobre as *Premises* que a compõem, apresentar um resultado verdadeiro, os respectivos *Methods* são ativados; (c) os *Methods* ativados alteram o valor de outro *Attribute*, completando assim o ciclo de execução do PON no PON-HD.

Um pequeno exemplo de uma aplicação em PON-HD é mostrado na Fig. 4. O circuito deste exemplo é um contador que conta até dez, quando uma entrada de controle está ativa. Em tempo, este circuito é equivalente a *Rule* apresentada no formato de regra se-então em alto nível na Fig. 1. Ademais, o atual desenvolvimento das implementações do PON já permite gerar diretamente a partir da chamada LingPON os circuitos equivalentes, o que permite expressar o sistema em alto nível. Este assunto será objeto de detalhamento na seção V.

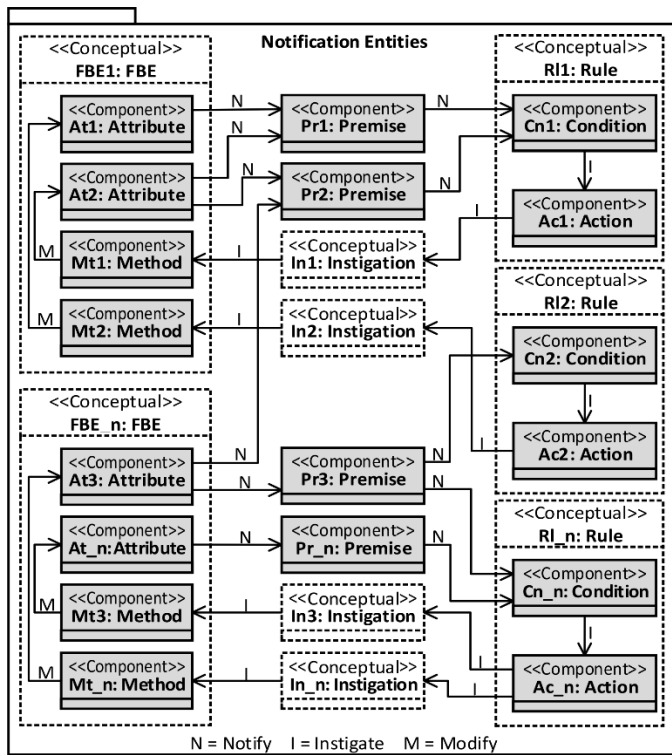


Figura 3. Diagrama de blocos do PON-HD

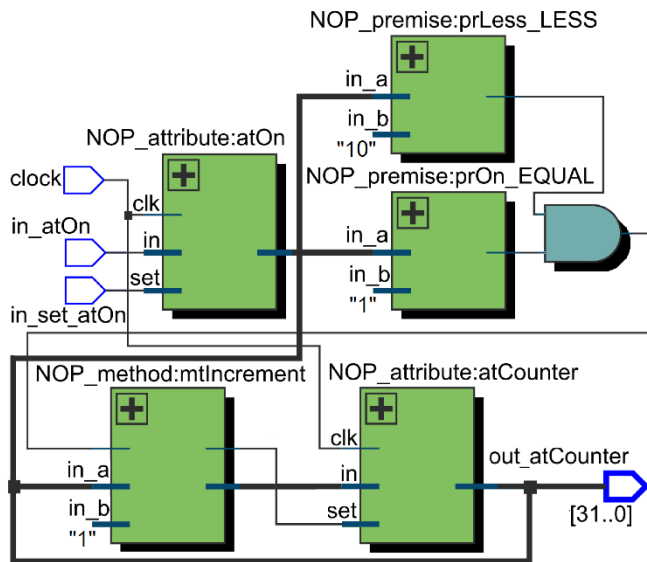


Figura 4. Diagrama esquemático da implementação em hardware.

A primeira *Premise* é chamada "prLess" e verifica se o *Attribute* "atCounter" é menor que a constante 10. Suas entradas são: o valor do *Attribute* "atCount" e a constante 10. A segunda *Premise* é chamada "prOn" e verifica se o valor do *Attribute* "atOn" é verdadeiro. As entradas são: o valor do *Attribute* "atOn" e a constante 1.

Há também um *Method* chamado "mtIncrement", cuja função é incrementar o valor do *Attribute* "atCounter". As suas entradas são: o valor do *Attribute* "atCounter" e a constante 1.

As entradas do circuito são: o sinal de *clock*, a entrada "atOn" que habilita a contagem e a entrada "Set_atOn" que armazena o valor da entrada no *Attribute* "atOn". A saída do

circuito é o valor armazenado no *Attribute* "atCount", que é composto de 32 bits.

A operação do circuito é simples. Quando um sinal de ativação é armazenado no *Attribute* "atOn", as *Premises* "prLess" e "prOn" tornam-se verdadeiras e ativam o *Method* "mtIncrement". Esse método incrementa o valor do *Attribute* "atCount" até atingir o valor 10. Neste ponto, a *Premise* "prLess" torna-se falsa, inibindo, assim o *Method* "mtIncrement" e parando a contagem.

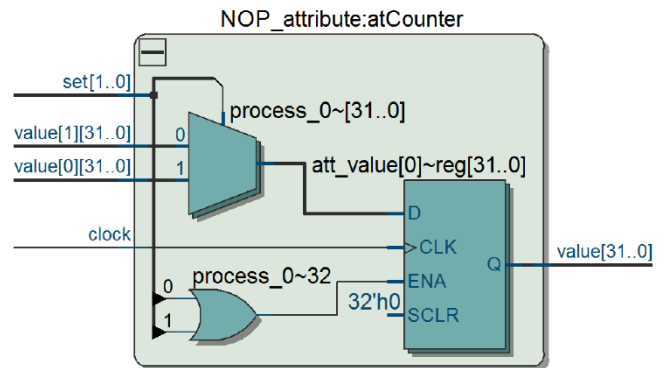
B. Os componentes do PON-HD

Para melhor compreensão do funcionamento do PON-HD será feito a seguir o detalhamento do funcionamento de cada um de seus componentes no *hardware*. Também serão apresentados detalhes sobre a implementação destes componentes, iniciando pelo *Attribute*.

Um *Attribute* é responsável por armazenar as informações no PON, portanto, é natural que no *hardware* o *Attribute* seja composto por registradores. O *Attribute* é o único elemento sincronizado pelo *clock* no PON-DH. Assim, todos os outros elementos são totalmente combinacionais. Essa estrutura permite que todo o ciclo de notificação do PON-HD ocorra em um único ciclo de *clock*.

Como o *Attribute* é implementado na forma de um componente VHDL, o número de bits que ele pode armazenar e o número de entradas que suporta são configuráveis por meio de parâmetros do tipo "generics". O conteúdo armazenado no *Attribute* é usado pelas *Premises* e pelos *Methods* em suas computações.

Os *Methods*, por sua vez, são capazes de atualizar o conteúdo dos *Attributes*, armazenando neles o resultado de suas computações e alimentando assim o ciclo de inferência. Como dois ou mais *Methods* podem acessar um determinado *Attribute* ao mesmo tempo, um mecanismo de prioridade foi implementado, neste mecanismo uma entrada de menor índice tem prioridade sobre uma entrada de maior índice (na Fig. 5 a entrada *value[0]* tem prioridade sobre a entrada *value[1]* e assim por diante). A Fig. 5 apresenta a estrutura de um *Attribute* de 32 bits com duas entradas.

Figura 5. Exemplo de *Attribute*.

Os *Methods* são responsáveis pelas computações realizadas no PON-HD. Qualquer circuito combinacional com duas entradas e uma saída pode ser implementado em um *Method*.

Na versão atual, são implementadas apenas operações aritméticas e lógicas simples.

Cada *Method* recebe duas entradas, que podem ser valores constantes ou valores de *Attributes*. A Fig. 6 mostra a estrutura de um *Method* que executa a operação de soma de valores de 32 bits.

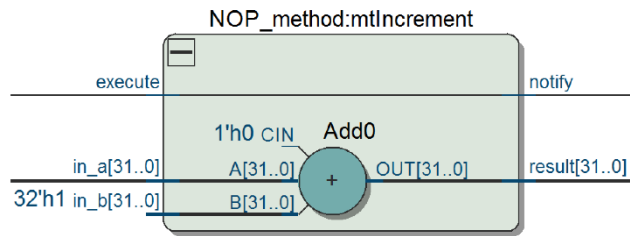


Figura 6. Exemplo de *method*.

Assim como os *Attributes*, os *Methods* são implementados como componentes VHDL. O número de bits das entradas e a operação a ser executada também são configuráveis por meio de "generics".

As *Rules*, assim como as *FBEs*, têm a função de agrupar os elementos e não têm nenhuma implementação em *hardware*. Uma *Rule* agrega *Actions* e *Conditions* e serve principalmente como uma abstração de alto nível que ajuda a organizar o projeto.

As *Premises* são os elementos responsáveis pelas operações relacionais no PON-HD. As operações relacionais implementadas são: igual, diferente, maior, maior ou igual, menor e menor ou igual. Uma *Premise* recebe duas entradas, que podem ser *Attributes* ou valores constantes. O resultado da *Premise* é um valor binário que indica se o condicional de teste é satisfeito ou não. A Fig. 7 mostra a estrutura de uma *Premise* que executa a operação "menor que" entre valores de 32 bits.

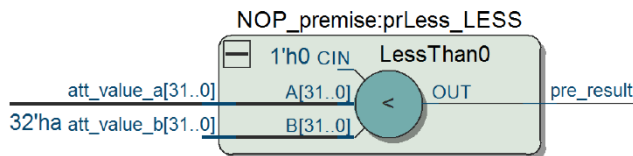


Figura 7. Exemplo de *Premise*.

A *Premise* também é implementada como um componente VHDL, que permite que o número de bits das entradas e a operação sejam configurados por meio de "generics".

Uma *Condition*, por sua vez, é responsável por executar uma operação lógica (E ou OU) entre os valores das *Premises*. Este componente é facilmente implementado em *hardware* através de portas E ou OU. Assim, não é necessário criar um componente VHDL. A porta E que aparece na Fig. 8 é um exemplo de implementação de uma *Condition*.

As *Actions* com as *Instigations* são responsáveis por transportar o sinal de ativação das *Conditions* para os *Methods*, seguindo as ligações estabelecidas no projeto. Assim, uma porta OU é suficiente para realizar essa tarefa no *hardware*, como mostrado na Fig. 8. Neste exemplo o *Method* "mtClear" é ativado quando a *Premise* "prOn" for verdadeira ou quando as *Premises* "prEnd" e "prLess" forem verdadeiras.

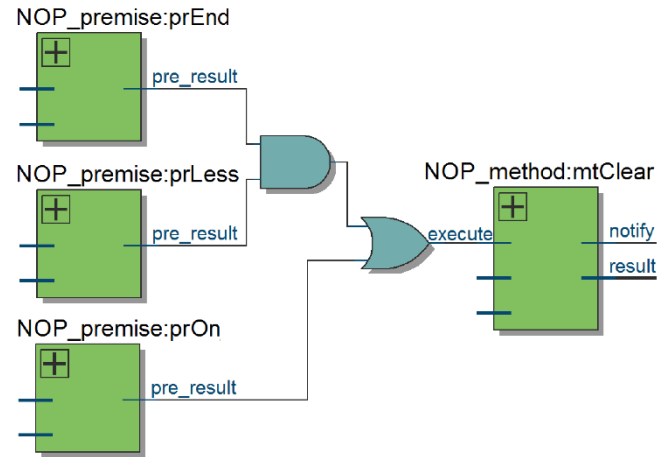


Figura 8. Exemplo de *Condition* e *Action*.

A implementação atual do PON-HD suporta operações com número inteiro e booleanos. No entanto, é fácil adicionar suporte para outros tipos de dados, alterando o código VHDL das *Premises* e dos *Methods*. Por exemplo, para adicionar suporte a ponto flutuante, é necessário alterar o código da *Premisse* de forma que seja possível realizar comparações entre números de ponto flutuante. Também é necessário alterar o código dos *Methods* para executar operações sobre este tipo de dados. Os *Attributes* não precisam ser alterados, pois eles apenas armazenam dados, independentemente de seu tipo.

É possível utilizar o PON-HD para criar *hardware* digital de duas formas distintas. Os componentes do PON-HD podem ser instanciados manualmente em um arquivo VHDL, onde o desenvolvedor é responsável por fazer todas as conexões e parametrizações. Outra forma de utilizar o PON-HD é através de um compilador. Neste caso o desenvolvedor cria a aplicação em uma linguagem de alto nível e o compilador gera automaticamente o arquivo VHDL com as conexões e parametrizações necessárias.

De qualquer forma o código VHDL resultante pode ser compilado, simulado e transferido para um FPGA com as ferramentas disponibilizadas pelo fabricante do *hardware*.

C. O PON-HD Como uma ferramenta de síntese de alto nível

O conhecimento embutido na aplicação em PON-HD é organizado na forma de fatos e de regras, portanto em um nível de abstração mais alto do que seria obtido em uma implementação diretamente em VHDL.

Sendo assim, o PON-HD fornece ao desenvolvedor uma abordagem de alto nível para resolver o problema da síntese de *hardware*. O desenvolvedor se preocupa apenas com o fluxo de notificações e os elementos do PON-HD e não com a complexa sintaxe das linguagens de descrição de *hardware*. Como os elementos do PON-HD são predefinidos e testados, a probabilidade de erros diminui.

A estrutura do projeto em PON-HD permite que o desenvolvedor se preocupe apenas com um elemento de cada vez. Isto porque os elementos do PON-HD são dissociados uns dos outros, sendo interligados apenas pelas notificações.

O PON-HD pode ser classificado de acordo com os critérios de classificação apresentados anteriormente [6][7]. Em relação à programação, PON-HD é derivado do PON e possui uma linguagem de programação específica. O PON pode ser usado em várias plataformas e para resolver muitos tipos de problemas, portanto, é uma solução genérica.

A facilidade de implementação é uma característica importante do PON-HD, uma vez que ele facilita a implementação dos circuitos por meio da utilização de elementos predefinidos. A modularidade também é adequada, porque a maior parte do código já está escrita nos arquivos que definem os elementos do PON-HD, aliviando assim o desenvolvedor da tarefa de escrever esse código.

Em relação aos tipos de dados suportados, atualmente o PON-HD suporta dados inteiros e booleanos. No entanto, é consideravelmente simples editar o código VHDL dos elementos do PON-HD para adicionar suporte para outros tipos de dados, tais como ponto flutuante. É possível afirmar que o PON-HD é independente da arquitetura, pois o resultado é um código VHDL completo que pode ser compilado para qualquer arquitetura ou fabricante.

Em termos de testes atualmente é possível verificar o funcionamento de um projeto em PON-HD por meio de um emulador, sem a necessidade de compilar e testar o projeto em uma FPGA. No que diz respeito à documentação do PON-HD é necessário ainda um pouco de trabalho, pois como o PON-HD ainda está em testes ele não possui uma documentação definitiva. A linguagem LingPON utilizada no PON-HD é fácil de aprender, porque exige que o desenvolvedor se habitue a usar um conjunto relativamente pequeno de elementos, com uma sintaxe simples.

A implementação atual do PON-HD é uma ferramenta interessante para resolver muitos tipos de problemas, mas não é adequado para todos os tipos de problemas. Devido à estrutura do PON-HD, a manipulação de fluxos de dados, como em filtros digitais ou de processamento de imagem, até é possível, mas é bastante complicada, pois como não estão implementados *Attributes* na forma de vetores a manipulação dos dados se torna complexa. O PON-HD é mais adequado para problemas de controle discreto, orientado a eventos, nos quais a mudança de estado de um elemento exige a tomada de decisões e a alteração do estado de outros elementos.

Finalmente, é importante salientar que, apesar do PON-HD utilizar regras na definição do comportamento do circuito, de forma similar ao que acontece no Bluespec, o princípio de funcionamento é bastante diferente. As regras do Bluespec são chamadas de ações atômicas e expressam o comportamento sob a forma de máquinas de estado finito (*FSMs – Finite State Machines*) cooperativas [7], enquanto que no PON-HD o comportamento é determinado por um circuito combinacional gerado dos *Attributes*, *Premises*, *Conditions* e *Methods*, sem o uso de *FSMs*.

V. O AMBIENTE DE DESENVOLVIMENTO DO PON-HD

Para permitir a utilização do PON no desenvolvimento de aplicações tornou-se necessário a criação de um ambiente de desenvolvimento. Este ambiente é composto por um

compilador e um emulador. Entende-se por compilador um programa que recebe como entrada um programa escrito em uma linguagem de programação, chamada linguagem fonte, e o traduz em um programa equivalente em outra linguagem chamada de linguagem alvo [16].

A linguagem LingPON e seu compilador foram apresentados em [16]. A versão original da LingPON e de seu compilador não são apropriadas para a criação de circuitos em hardware. Desta forma foi desenvolvida uma linguagem específica para o PON-HD, chamada LingPONHD. Também foi desenvolvida uma nova versão do compilador com suporte a esta linguagem e com capacidade de gerar código VHDL. A linguagem LingPONHD e seu compilador são descritos neste artigo, e são apresentados a seguir.

A. A linguagem do PON-HD

A Fig. 9 apresenta um programa em LingPONHD. Este é o programa que gerou o circuito da Fig. 4.

```
// exemplo de programa em lingPON
entity Contador

attribute integer atCounter 0 out
attribute boolean atOn 0 in

premise prLess atCounter < 10
premise prOn atOn == 1

rule rlCont mtIncrement prOn and prLess

method mtIncrement atCounter = atCounter + 1
```

Figura 9. Exemplo de programa em LingPONHD.

Neste exemplo é possível identificar as entidades do PON como *Attributes*, *Premises*, *Rules* e *Methods* bem como as relações entre elas e seus parâmetros. Também é possível declarar elementos que não pertencem ao PON original, mas são necessários ao código VHDL resultante da compilação, como é o caso da declaração *entity*, onde é fornecido o nome da entidade, e das declarações *in* e *out* que determinam se um *Attribute* é usado como entrada ou saída do circuito.

B. O compilador PON-HD

Um compilador pode ser visto como um programa tradutor que transforma uma linguagem fonte em uma linguagem objeto geralmente de mais baixo nível. No caso do PON-HD a linguagem fonte é o LingPONHD e a linguagem objeto é VHDL.

O código VHDL gerado faz uso de um conjunto de componentes declarados em arquivos VHDL separados. Estes arquivos contêm a descrição de cada um dos componentes do PON-HD. O conjunto de arquivos VHDL resultante pode ser então compilado e simulado por ferramentas disponíveis no mercado para este fim.

O compilado PON-HD também é responsável por gerar um arquivo contendo as estruturas de dados necessárias ao funcionamento do emulador PON-HD.

C. O emulador PON-HD

Quando se está desenvolvendo aplicações em PON-HD,

onde a execução de todos os elementos acontece de forma paralela, nem sempre é fácil prever como estes elementos irão interagir. Assim para facilitar a depuração destas aplicações foi desenvolvido um emulador PON-HD. Este emulador é um aplicativo de computador, com uma interface gráfica que permite visualizar o comportamento de cada um dos elementos do programa em LingPON. A Fig. 10 apresenta a interface do emulador PON-HD.

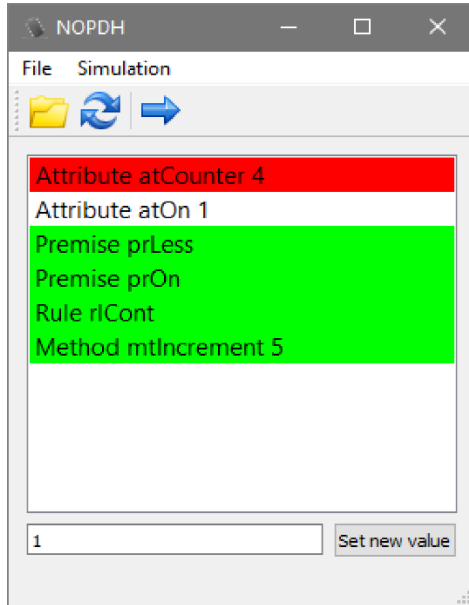


Figura 10. Emulador PON-HD.

Este emulador recebe como entrada um arquivo contendo estruturas de dados que descrevem o comportamento do programa. Estas estruturas de dados são geradas pelo compilador PON-HD, durante o processo de compilação.

O emulador permite executar passo a passo (ciclo de *clock*) o programa em PON-HD. Durante esta execução é possível visualizar o valor de cada um dos *Attributes* e o resultado de cada um dos *Methods* numericamente. Os resultados das *Premises*, das *Rules* e o estado de ativação dos *Methods* também pode ser visualizado através de um código de cores. É possível ainda alterar o valor dos *Attributes* de forma a emular entradas ou testar situações específicas.

Este emulador é uma ferramenta muito importante pois permite verificar o comportamento do programa sem a necessidade de compilar o código VHDL gerado, uma vez que este é normalmente é um processo demorado e exige ferramentas específicas.

VI. EXPERIMENTOS REALIZADOS

Para comprovar o funcionamento dos elementos que compõem o PON-HD foram realizados alguns experimentos, dos quais três serão descritos a seguir. O primeiro experimento é um contador que conta até dez quando uma entrada de controle é habilitada. O segundo exemplo é um ordenador que ordena um conjunto variável de números inteiros de 16 bits em ordem crescente. E finalmente o terceiro experimento é um controlador para um robô

hexápode.

A. Experimento do contador

Este experimento, apesar de bastante simples, serve para demonstrar como o PON-HD funciona. Este pequeno projeto já foi utilizado anteriormente neste texto como exemplo. A Fig. 4 apresenta o circuito gerado após a compilação do código VHDL e a Fig. 9 apresenta o código em LingPONHD para este experimento. O circuito deste experimento possui uma entrada de *clock* denominada “*clock*”, uma entrada de controle que habilita a contagem chamada “*in_atOn*” e uma entrada que armazena o valor de “*in_atOn*” no *Attribute* “*atOn*” chamada “*in_set_atOn*”. Este circuito possui apenas uma saída, que é o valor da contagem, armazenado no *Attribute* “*atCounter*” e é chamada “*out_atCounter*”. O funcionamento é o seguinte: quando a valor do *Attribute* “*atOn*” é 1 (verdadeiro) e o valor do *Attribute* “*atCounter*” é menor que dez as *Premises* “*prOn*” e “*prLess*” se tornam verdadeiras e notificam o *Method* “*mtIncrement*” que por sua vez incrementa o valor do *Attribute* “*atCounter*”. Assim que umas das condições se tornar falsa a contagem é interrompida.

B. Experimento do ordenador de dados

Um experimento com um número muito maior de elementos foi desenvolvido para testar o desempenho do PON-HD se comparado à codificação realizada diretamente em VHDL. Neste experimento, conjuntos de números inteiros de 16 bits são armazenados nos *Attributes* e o objetivo é classificá-los em ordem crescente. A Fig. 11 apresenta um diagrama de blocos onde são representadas as entidades do PON-HD conforme empregadas neste experimento.

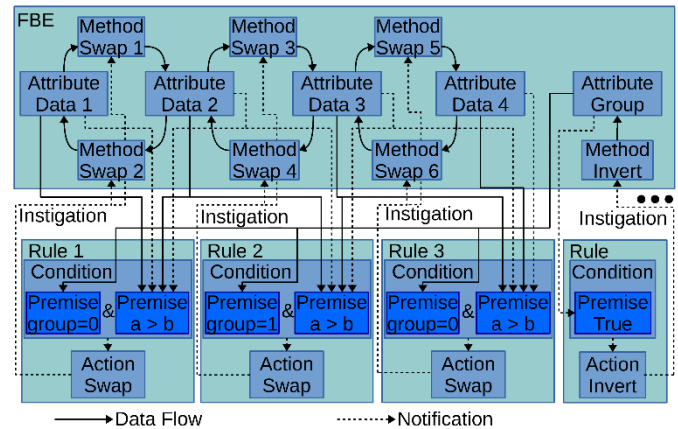


Figura 11. Diagrama de blocos do ordenador PON.

O algoritmo de ordenação empregado é o “Odd-Even Sort” [19]. Neste algoritmo, todos os pares com índices (ímpar, par) de elementos adjacentes no conjunto de dados são comparados e, se um par está na ordem errada (o primeiro é maior do que o segundo), os elementos são trocados. O mesmo acontece, na sequência, para os pares com índices (par, ímpar). As etapas (ímpar, par) e (par, ímpar) são repetidas até que o conjunto de dados esteja completamente ordenado.

Na Fig. 11 são apresentados apenas quatro elementos, mas foram realizados experimentos com conjuntos de até três mil

elementos, desta forma pôde-se avaliar o comportamento do PON-HD para diferentes tamanhos de circuitos.

Neste experimento tem-se um *Attribute* que representa quais elementos estão sendo comparados, os pares ou os ímpares. Um conjunto de *Premise* e *Method* se encarrega de alternar entre o grupo dos elementos pares e o grupo dos elementos ímpares. Os dados a serem ordenados também estão armazenados em *Attributes*, para cada par de dados existe uma *Condition* que por sua vez é composta por duas *Premises*. As *Premises* são: o primeiro elemento do par é maior que o segundo (o par está fora da ordem desejada) e o grupo que está sendo comparado é o grupo desejado (par ou ímpar). Se as duas condições forem satisfeitas os *Methods* responsáveis pela troca dos dados são executados. Após um determinado número de repetições, que vai depender dos dados, a ordenação é finalizada.

A título de comparação o mesmo algoritmo de ordenação foi implementado também em VHDL, sem a ajuda do PON-HD. Desta forma foi possível comparar o esforço necessário para implementar este algoritmo, no *hardware*, com e sem a ajuda das ferramentas do PON-HD. Também foram feitas comparações entre as duas implementações com relação ao desempenho.

C. Experimento do controlador de robô hexápode

Para verificar a confiabilidade do PON-HD um experimento mais complexo foi elaborado. Neste experimento foi desenvolvida a lógica de controle de um robô hexápode. A Fig. 12 apresenta o robô deste experimento.

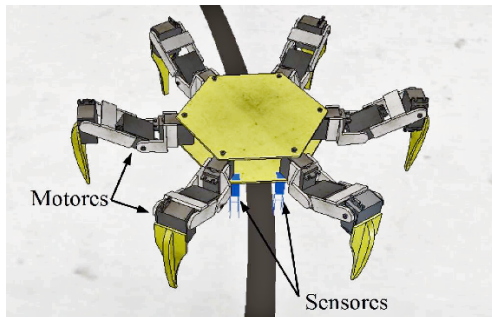


Figura 12. Robô hexápode.

Este robô é composto por seis pernas, cada uma com três motores, totalizando assim dezoito motores a serem controlados. O robô também possui dois sensores óticos capazes de identificar a cor da superfície onde o robô está andando.

O controlador é responsável por calcular a posição angular de cada um dos dezoito motores de forma a fazer o robô caminhar acompanhando uma linha preta desenhada na superfície onde o robô se encontra. O robô é capaz de determinar sua posição em relação à linha preta utilizando seus sensores. As entradas do controlador são dois sinais que representam a cor da superfície sob cada um dos sensores. A saída é a posição angular de cada um dos dezoito motores.

Para permitir o sincronismo dos motores e assim realizar os movimentos do robô foi implementada uma espécie de máquina de estados em LingPONHD. Para tanto um *Attribute*

foi utilizado para contar e sincronizar as etapas dos movimentos.

O controlador foi implementado em LingPONHD e o circuito resultante da compilação foi gravado em uma FPGA. O robô foi simulado em um computador por meio do simulador de robôs V-Rep [20]. A conexão entre a FPGA com o controlador e o computador onde o robô foi simulado se deu por meio de um link de comunicação de dados Bluetooth. A Fig. 13 apresenta o *hardware* utilizado no experimento.

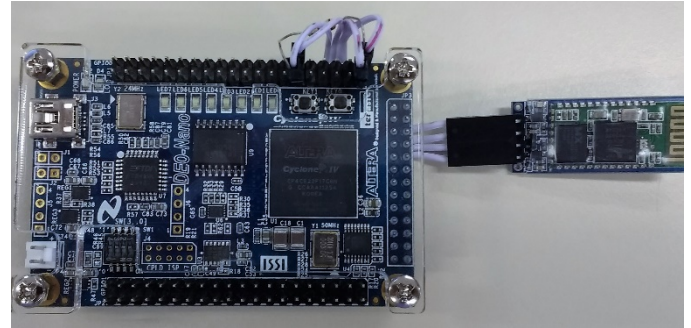


Figura 13. Hardware utilizado no experimento do robô.

A placa utilizada é uma DE0-Nano que utiliza uma FPGA *Cyclone IV* da Altera. O módulo de comunicação Bluetooth encontra-se a direita.

O controlador do robô hexápode, foi implementado em LingPONHD utilizando 29 *Attributes*, 50 *Premises*, 127 *Rules*, e 103 *Methods*. A Fig. 14 apresenta um trecho deste programa em LingPONHD.

```

premise prAndaSd sensorDir == 0
premise prAndaSe sensorEsq == 0
rule rIAnda mtAnda prAndaSd and prAndaSe

premise prViEsSd sensorDir == 0
premise prViEsSe sensorEsq == 1
rule rLViraEsquerda mtViraEsquerda prViEsSd and prViEsSe

premise prViDiSd sensorDir == 1
rule rLViraDireita mtViraDireita prViDiSd
  
```

Figura 14. Trecho do programa de controle do robô.

Neste trecho é processado o sinal dos sensores que detectam a linha. A decisão sobre qual a direção que o robô deve seguir é tomada com base no sinal destes sensores.

VII. RESULTADOS

O circuito contador do primeiro experimento mostrou como é fácil programar em LingPONHD e que o circuito gerado no *hardware* corresponde fielmente ao que foi programado como pode ser visto na Fig. 4. Este diagrama foi obtido com a ferramenta “RTL Viewer” da Altera [21], representando assim o resultado da compilação do código VHDL.

O circuito ordenador de dados comportou-se conforme esperado. A Fig. 15 apresenta o comportamento de um conjunto de dados aleatórios de dezesseis elementos durante o processo de ordenação.

controlador em LingPONHD não foi complicado. O desacoplamento entre os elementos do PON-HD e o paralelismo na execução das regras propiciado pela FPGA facilitaram este processo.

VIII. CONCLUSÃO

O PON vem apresentando resultados positivos em várias plataformas, no entanto, a única implementação que reproduz com precisão o modelo conceitual paralelo do PON é o PON-HD. A implementação do PON em *hardware* permite que todas as notificações aconteçam em paralelo, maximizando o desempenho do sistema.

Por meio dos experimentos, foi possível observar que os circuitos criados com a ajuda do PON-HD funcionam corretamente. Foi também possível observar que as implementações em PON-HD e VHDL tem características de desempenho muito semelhantes, demonstrando que a utilização do PON-HD não impacta no desempenho do circuito. O número de elementos lógicos utilizados por cada um dos circuitos ordenadores ficou praticamente igual em ambas as implementações, demonstrando que a utilização do PON-HD não aumenta o consumo de *hardware*. O PON-HD também gerou circuitos com uma frequência de operação igual e por vezes até superior aos circuitos criados diretamente em VHDL.

Os resultados mostram que o PON-HD é uma alternativa interessante, pois facilita a criação dos circuitos fornecendo um melhor nível de abstração, sem comprometer o desempenho em termos de frequência de operação e latência de execução e sem aumentar o consumo de elementos lógicos.

Como o PON-HD faz uso de uma linguagem simples e clara, empregando componentes previamente testados, fica mais fácil criar os circuitos e a possibilidade de erros é reduzida. Assim, pode-se concluir que o PON-HD pode ser utilizado como uma ferramenta de síntese de alto nível.

AGRADECIMENTOS

Os autores agradecem à Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) pelo apoio financeiro.

REFERÊNCIAS

- [1] R. Crosbie, "Using field-programmable gate arrays for high-speed real-time simulation," *International Journal of Modeling, Simulation, and Scientific Computing*, vol. 1, no. 01, pp. 99–115, 2010.
- [2] J. Qian, H. Xu, Y.-S. DONG, X. Liu, and Y. Wang, "FPGA acceleration window joins over multiple data streams," *Journal of Circuits, Systems, and Computers*, vol. 14, no. 04, pp. 813–830, 2005.
- [3] S. Borkar and A. A. Chien, "The future of microprocessors," *Communications of the ACM*, vol. 54, no. 5, pp. 67–77, 2011.
- [4] D. Olds, "Are FPGAs the answer to the Compute Gap," *insideHPC*, 2016.
- [5] M. Awad, "FPGA supercomputing platforms: a survey," in *Field Programmable Logic and Applications*, 2009. FPL 2009. International Conference on, 2009, pp. 564–568.
- [6] W. Meeus, K. Van Beeck, T. Goedemé, J. Meel, and D. Stroobandt, "An overview of today's high-level synthesis tools," *Design Automation for Embedded Systems*, vol. 16, no. 3, pp. 31–51, 2012.
- [7] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, and others, "A Survey and Evaluation of FPGA High-Level Synthesis Tools."

- [8] J. M. Simão and P. C. Stadzisz, "Paradigma Orientado a Notificações (PON)—Uma Técnica de Composição e Execução de Software Orientado a Notificações," *Pedido de Patente submetida ao INPI/Brazil (Instituto Nacional de Propriedade Industrial) em*, 2008.
- [9] J. M. Simão and P. C. Stadzisz, "Inference based on notifications: a holonic metamodel applied to control issues," *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, vol. 39, no. 1, pp. 238–250, 2009.
- [10] J. M. Simão, P. C. Stadzisz, C. E. Lima, F. A. Witt, and R. R. Linhares, "Paradigma Orientado a Notificações em Hardware Digital," *Pedido de Proteção Industrial e Pedido de Patente enviados à Agência de Inovação da UTFPR respectivamente em*, vol. 11, no. 05, 2012.
- [11] G. Gupta and G. S. Sohi, "Dataflow execution of sequential imperative programs on multicore architectures," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 59–70.
- [12] S. Windh, X. Ma, R. J. Halstead, P. Budhkar, Z. Luna, O. Hussaini, and W. A. Najjar, "High-Level Language Tools for Reconfigurable Computing," *Proceedings of the IEEE*, vol. 103, no. 3, pp. 390–408, 2015.
- [13] M. Gabbriellini and S. Martini, *Programming Languages: Principles and Paradigms*. Springer Science & Business Media, 2010.
- [14] R. R. Linhares, D. P. B. Renaux, J. M. Simão, and P. Cezar Stadzisz, "Evaluation of the Notification Oriented Paradigm Applied to Real-Time Systems," in *Computing Systems Engineering (SBESC)*, 2014 Brazilian Symposium on, 2014, pp. 132–137.
- [15] J. M. Simão, D. P. Renaux, R. R. Linhares, and P. C. Stadzisz, "Evaluation of the Notification Oriented Paradigm Applied to Sentient Computing," in *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*, 2014, pp. 253–260.
- [16] C. A. Ferreira, "Linguagem e compilador para o paradigma orientado a notificações (PON): avanços e comparações," 2015.
- [17] E. Peters, R. P. Jasinski, V. A. Pedroni, and J. M. Simão, "A new hardware coprocessor for accelerating Notification-Oriented applications," in *Field-Programmable Technology (FPT)*, 2012 International Conference on, 2012, pp. 257–260.
- [18] R. R. Linhares, J. M. Simão, and P. C. Stadzisz, "NOCA - A Notification-Oriented Computer Architecture," *IEEE Latin America Transactions*, vol. 13, no. 5, pp. 1593–1604, 2015.
- [19] B. Mozaffari, "Optimization of Odd-Even Transposition Network," in *2010 2nd International Conference on Education Technology and Computer*, 2010.
- [20] E. Rohmer, S. P. N. Singh, M. Freese, "V-REP: a Versatile and Scalable Robot Simulation Framework", *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, 2013.
- [21] ALTERA CORPORATION. Introduction to Quartus II Software. URL: https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/manual/quartus2_introduction.pdf



Ricardo Kerschbaumer was born in Joaçaba - SC, Brazil, in 1975. He received in 2002 his B.Sc. in Electrical Engineering, with emphasis on Electronics, from Santa Catarina State University (UDESC), Brazil. He received in 2008 his M.Sc degree in Industrial Informatics from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He is currently Ph.D. student in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, as well as Professor with the Federal Institute of Education, Science and Technology (IFC), teaching in the course of control and automation engineering. His interests include electronics, programmable logic devices and microcontrollers.



Jean Marcelo Simão was born in Ponta Grossa - Paraná, Brazil, in 1976. He received in 1994 the Technical degree in informatics from State School of Paraná (CEP) and in 1998 the B.Sc. degree in computer science from State University of Ponta Grossa (UEPG). In 2001, he obtained the M.Sc. degree from Graduate School in Electrical Engineering and

Industrial Computer Science (CPGEI) at Federal Center for Technological Education of Parana (CEFET-PR), situated in Curitiba (Brazil), nowadays named as Federal University of Technology - Paraná (UTFPR). On June 2005, he obtained the Ph.D. degree, after a double thesis, in the domains of industrial computer science at CPGEI/UTFPR and computer engineering & automatics at Research Center for Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP), situated in France. After, in 2006, he developed teaching activities in a Master at UHP and researches ones at CRAN in a post-doctoral context. Nowadays, he is Associated Professor at UTFPR with affiliation to the Department of Electronics and collaboration with the Departments of Informatics. He is also affiliated to the Laboratory of Innovation in Technology (LIT) of the Centre of Technology Innovation (CITEC) at UTFPR. In 2009, he was affiliated to Graduate School in Applied Computer Science PPGCA from DAINF/UTFPR. Also in 2010, he was affiliated to UTFPR. Finally, his teaching activities concern computer science to engineering courses, whereas his publications and current research interests include agile/holonic manufacturing, artificial intelligence, software/system engineering, computer science, and programming and computer paradigms.



João Alberto Fabro is an Associate Professor at Federal University of Technology - Parana (UTFPR), where he works since 2008. From 1998 to 2007, he was with the State University of West-Parana (UNIOESTE). He has an undergraduate degree in Informatics, from Federal University of Paraná (UFPR 1994), a Master's Degree in Computing

and Electric Engineering, from Campinas State University (UNICAMP 1996), a Ph.D. degree in Electric Engineering and Industrial Informatics (CPGEI) from UTFPR (2003) and recently actuated as a Post-Doc at the Faculty of Engineering, University of Porto, Portugal (FEUP, 2014). Has experience in Computer Science, especially Computational Intelligence, actively researching on the following subjects: Computational Intelligence (neural networks, evolutionary computing and fuzzy systems), and Autonomous Mobile Robotics. Since 2009, has participated in several Robotics Competitions, in Brazil, Latin America and World Robocup, both with soccer robots and service robots.



Carlos Raimundo Erig Lima was born in Ijuí, Rio Grande do Sul, Brazil, in 1964. He received in 1989 his B.Sc. in Electrical Engineering, with emphasis on Electronics and Telecommunications, from Federal University of Technology - Paraná (UTFPR), Brazil, which is formerly known as Federal Center for Technological Education of Parana. He

received in 1994 his M.Sc degree from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, and in 2003 his Ph.D. degree in Mechanical Engineering from Campinas State University (UNICAMP). He is currently Full Professor at University of Technology of Paraná, teaching in the undergraduate program in Electrical Engineering and in the graduate program in Electrical Engineering, carrying out activities of teaching, research and extension. Has experience in Electrical Engineering, with emphasis on electronic process control and feedback, working mainly on the following themes: Reconfigurable computing, interfaces and control systems, mobile robotics, embedded systems design, and bioinformatics.



Robson Ribeiro Linhares was born in Curitiba - PR, Brazil, in 1975. He received in 1999 his B.Sc. in Electrical Engineering, with emphasis on Electronics and Telecommunications, from Federal University of Technology - Paraná (UTFPR), Brazil, which is formerly known as Federal Center for Technological Education of Parana. He

received in 2001 his M.Sc degree in software engineering for real-time systems from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, and in 2015 his Ph.D. degree in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He is Professor at the UTFPR, teaching in a set of educational programs such as the undergraduate courses of Computer Engineering and Information Systems and the Graduate Program in Applied Computing (PPGCA). His research interests include software engineering, programming languages and paradigms, embedded systems and computer architectures.